

Penney's Game and Conway's Formula

Part II — A Full Derivation and Crash Log

From the reliable joint-Markov-chain solution, to a self-built recursion that crashed completely, to the matrix closed form and the martingale; and how multiple AIs cross-checking each other plus a code referee rescued a

conclusion that memory had quietly led astray

Read this first: what this document is

This is **not** a clean “textbook answer.” It is an honest **exploration log**: how we tried to derive the odds formula for Penney's game from first principles, **walked into a dead end** for seven or eight hours, got it refuted by one line of code, and then — with three AIs (Claude, GPT, DeepSeek) poking holes in each other plus a Python joint chain acting as “referee” — located the error and found the correct solution.

We **deliberately keep every wrong turn**, including the full episode where Claude gave a wrong formula from memory and insisted it was “obvious,” only to be refuted on the spot by a two-character counterexample. Because **understanding why a derivation goes wrong is far more valuable than the final answer**. Textbooks hand you the polished conclusion; this log hands you all the mud the conclusion was born in.

Reading guide. Four colored boxes throughout:

- **Purple / Intuition**: before any formula, the plain-language “why it should be this way.”
- **Gray / Attempt**: a step we actually wrote (right or wrong).
- **Orange / Crash**: where that step is wrong and what refuted it.
- **Green / Click**: what we genuinely understood after the correction.

Contents

1	Warm-up: The Rules and Two Counterintuitive Questions	2
1.1	The game	2
1.2	Two questions that trip people up	2
2	The Reliable Solution: The Joint Markov Chain	3
2.1	Why the state must track “both progresses at once”	3
2.2	A mismatch “rewinds” : the KMP fallback	3
2.3	Worked example: HHH vs THH	4
2.3.1	Step 1: list the “undecided” states	4
2.3.2	Step 2: where does each state go on H / on T	4
2.3.3	Step 3: write win-probability equations	5

2.3.4	Step 4: solve	5
3	The Crash Log: A Dead End in Search of a “Closed Form”	6
3.1	Detour 1: Is $P(\text{length} = n) = \frac{1}{2^n}$?	6
3.2	Detour 2: The survival term forgot the opponent Y	7
3.3	Detour 3: The (k, p) partition is not mutually exclusive (the “compatible” trap)	7
3.4	Detour 4: A “pretty-looking” self-built recursion	8
3.5	Detour 5: Claude gives a false formula from memory, and calls it “obvious”	8
4	The Verification Crisis: A Code Referee Settles It	9
4.1	Testing with the referee: an unequal-length example	9
4.1.1	The referee gives the true value	9
4.1.2	What our recursion gives	10
5	The Correct Solution: State, Matrix Closed Form, and the Martingale	10
5.1	Error 1: $\Gamma_m = j$ is not a sufficient state	10
5.2	Error 2: role rotation mistakes “progress” for a “new pattern”	11
5.3	A picture: the joint chain for HH vs HTH	11
5.4	The matrix closed form: one formula for equal and unequal length	11
5.4.1	The safe-transition matrix M	12
5.4.2	Counting prefixes: $N_m = e_0 M^m$	12
5.4.3	The final-sprint filter R_X : replacing the wrong χ_j	12
5.4.4	Putting it together: the closed form	12
5.5	Where exactly the martingale is	13
5.5.1	Three easily-confused objects	13
5.5.2	F satisfies Bellman; $F(S_{t \wedge \tau})$ is a strict martingale	13
5.5.3	Odds is the ratio of two martingales’ initial values	14
6	Three Cases and That Pretty Conway Formula	14
6.1	Case 1: identical patterns	14
6.2	Case 2: equal length, different — the pretty Conway formula	15
6.3	Case 3: unequal length — do not force the Conway difference	15
6.4	Aside: the discarded K vs. the true Conway C	16
7	Back to Question 2: No Strongest Pattern (Non-transitivity)	16
8	Meta-Lesson: Approaching Truth When Every Source Errs	16
8.1	The full “crash-and-rescue” chain	16
8.2	Four transferable disciplines	17
8.3	The mathematics that actually settled	17
8.4	Directions for a Part III	18

1 Warm-up: The Rules and Two Counterintuitive Questions

1.1 The game

Flip a **fair coin**: heads H, tails T, each with probability $\frac{1}{2}$, forever, producing an infinite sequence such as

HTTHTHHT . . .

Two players each fix a **pattern** — a short string of H's and T's. Say player A picks HHH and player B picks THH. As the sequence grows, **whoever's pattern appears first as a contiguous substring wins**.

In plain language

Imagine two people listening to the same Morse stream. A waits for “dot-dot-dot,” B for “dash-dot-dot.” Symbols arrive one at a time; the first to hear their own string wins. Crucially both **listen to the same stream** — this is the source of every subtlety in the problem: the symbols you emit while building your own pattern simultaneously help (or don't help) your opponent build theirs.

Write the random “ending position of the first occurrence of X ” as

$$T_X = \min\{t : \omega_{t-|X|+1} \cdots \omega_t = X\},$$

so “ X before Y ” is the event $\{T_X < T_Y\}$, and the central quantity is

$$V(X, Y) = \mathbb{P}(T_X < T_Y) \quad (\text{probability that “} X \text{ wins first”}).$$

1.2 Two questions that trip people up

Question 1 (how to compute): Given X, Y , how do we compute $V(X, Y)$? Is it the same thing as “how long on average until you first see X ,” i.e. the expectation $\mathbb{E}[T_X]$?

Question 2 (is there a strongest): Is there a “best” pattern that beats everyone? Equivalently, is “pick the pattern with the shortest expected wait” a winning strategy?

Two seeds planted by Part I

In Part I (waiting for HHH / HTH) we found: the average wait for HHH is $\mathbb{E}[T_{\text{HHH}}] = 14$, while for HTH it is only $\mathbb{E}[T_{\text{HTH}}] = 10$. Both patterns have **length 3** and each specific triple has probability $\frac{1}{8}$, yet the average waits differ by 4. The entire reason is **self-overlap**: HHH overlaps heavily with itself, so a broken match “rewinds” far, making it slow.

Remember the word **self-overlap**. In this part we'll see that short expected wait $\neq$ high duel win-rate, and that self-overlap is the engine behind the strange fact that there is no strongest pattern.

A spoiler that breaks intuition

You might think: HHH and THH are both length 3, so the duel should be roughly 50–50, right? **Badly wrong**. We will prove THH crushes HHH by **7:1**. Why? Because once any T shows up, HHH is essentially doomed — that T instantly becomes the start of THH. Hold this spoiler; Section 2 turns it into a precise $\frac{1}{8} : \frac{7}{8}$.

2 The Reliable Solution: The Joint Markov Chain

This chapter gives the **one solution in this whole log that is correct end-to-end, works for any patterns (including unequal length), and is verified repeatedly by code**. Later we derive a “prettier” self-built formula that crashes — and this chapter is the **referee** that judges it. So learn it well.

2.1 Why the state must track “both progresses at once”

What do you actually need to remember to play this?

Suppose you’ re scanning the sequence, trying to judge “will A (X) or B (Y) complete first.” What you really need to remember is not the long history, but **how close each pattern currently is to completion**.

E.g. targets $X = HH$, $Y = HTH$. If the current suffix is H , then: for $X = HH$ I’ ve laid down one H (progress 1); for $Y = HTH$ I’ ve also laid down its opening H (progress 1). **These “two progresses” are all you need** — earlier history has no bearing on who wins next. That is the “memoryless” (Markov) property.

So we define the **state** as a **progress pair**

$$s = (i, j), \quad \begin{cases} i = \text{longest match of current suffix to a prefix of } X \text{ (“} X \text{ is at cell } i \text{”)}, \\ j = \text{longest match of current suffix to a prefix of } Y \text{ (“} Y \text{ is at cell } j \text{”)}. \end{cases}$$

Each new coin updates i and j simultaneously. If either reaches full length ($i = |X|$ or $j = |Y|$), that side **wins** and the game stops in an **absorbing state**.

The one essential difference from the single pattern (Part I)

Waiting for a single HHH , the state was “one progress.” Here **two patterns fight over the same stream**, so the state tracks “two progresses” (i, j) , and they **share every coin** — the same character may advance one, or advance/rewind the other. This sharing and coupling is what makes two patterns harder than one, and it is exactly what dooms us later the moment we track only one progress.

2.2 A mismatch “rewinds” : the KMP fallback

Why progress rewinds rather than resetting to zero

Target $X = HHH$, currently at progress 2 (suffix HH). A T arrives: HHT can’ t continue HHH , so progress drops to 0. But take $X = HH$ with $Y = HTH$: if Y is at HT (progress 2) and an H arrives, it completes HTH — Y wins; if a T arrives we get HTT , whose trailing T is not Y ’ s opening H , so progress falls back to 0.

The point: **on a mismatch, progress drops not necessarily to 0 but to “how much the current suffix still matches the pattern’ s prefix.”** Where it lands is dictated by the pattern’ s own structure — this is the KMP failure function. You needn’ t prove KMP; just accept that every (progress, new-char) pair uniquely determines a new progress, obtainable from a table.

We write these tables as $\delta_X(i, c)$ and $\delta_Y(j, c)$: given current progress and the new char c , return the new progress. After reading c , state (i, j) becomes $(\delta_X(i, c), \delta_Y(j, c))$; if a new progress hits full length, we enter the corresponding absorbing state.

2.3 Worked example: HHH vs THH

We now compute the 7:1 from the spoiler. Let $A = HHH$, $B = THH$.

2.3.1 Step 1: list the “undecided” states

Name each state by the **relevant current suffix** (which uniquely fixes (i, j)). The reachable transient states are five:

$$a_0 = (\emptyset), a_1 = (\text{H}), a_2 = (\text{HH}), b_1 = (\text{T}), b_2 = (\text{TH}),$$

plus two absorbing states $A = (\text{HHH wins}), B = (\text{THH wins})$.

Why only 5 transient states, not a huge number

You might fear a state explosion. But the current suffix **cannot simultaneously** match HHH deeply and THH deeply — they start differently (H vs T). **Many “combinations” are simply impossible**, so few states are reachable. “Which combos are impossible” is self-/cross-overlap quietly at work — hold that feeling; it returns formally as Conway’s correlation number.

2.3.2 Step 2: where does each state go on H / on T

Work it out one by one (tedious, but confirm each):

- $a_0 = (\emptyset)$: H $\rightarrow a_1$; T $\rightarrow b_1$.
- $a_1 = (\text{H})$: H $\rightarrow \text{HH} = a_2$; T $\rightarrow b_1$.
- $a_2 = (\text{HH})$: H $\rightarrow \text{HHH}$, **A wins** = A ; T $\rightarrow b_1$.
- $b_1 = (\text{T})$: H $\rightarrow \text{TH} = b_2$; T $\rightarrow$ still T = b_1 (**self-loop!**).
- $b_2 = (\text{TH})$: H $\rightarrow \text{THH}$, **B wins** = B ; T $\rightarrow b_1$.

That self-loop $b_1 \xrightarrow{\text{T}} b_1$ is the soul of the problem

At $b_1 = (\text{T})$ reading T: the suffix goes $\text{T} \rightarrow \text{TT}$, but only the last T matters for $B = \text{THH}$, so we stay at “one T laid down” = b_1 . **This self-loop means: once you fall into state T, no matter how many T’s you flip you stay there, holding B’s opening locked in.** That’s why $B = \text{THH}$ is so strong — its opening T is a sticky trap, and the opponent $A = \text{HHH}$ is finished the moment a T appears.

2.3.3 Step 3: write win-probability equations

Let $h_s = \mathbb{P}(\text{starting from } s, A \text{ wins first})$. Two boundary values are obvious:

$$h_A = 1 \text{ (A already won)}, \quad h_B = 0 \text{ (B won, A has no chance)}.$$

Why there is “no +1” — the easiest confusion with Part I

Computing an **expected number of steps** in Part I, each step adds 1 (you’re counting steps): $e_s = 1 + \frac{1}{2}e_{s'} + \frac{1}{2}e_{s''}$. Here we compute a **probability**, not steps! The win-rate from s is the weighted average of “win-rate after H” and “win-rate after T,” with **no +1**:

$$h_s = \frac{1}{2} h_{(\text{state after H})} + \frac{1}{2} h_{(\text{state after T})}.$$

In one line: counting steps has +1; computing a probability has no +1. Keeping these straight is the key to not slipping in this chapter.

Written out (using Step 2' s transitions):

$$\begin{aligned}
 h_{b_1} &= \frac{1}{2}h_{b_2} + \frac{1}{2}h_{b_1} & (b_1 : H \rightarrow b_2, T \rightarrow b_1 \text{ self-loop}) \\
 h_{b_2} &= \frac{1}{2}h_B + \frac{1}{2}h_{b_1} = \frac{1}{2} \cdot 0 + \frac{1}{2}h_{b_1} & (b_2 : H \rightarrow B, T \rightarrow b_1) \\
 h_{a_2} &= \frac{1}{2}h_A + \frac{1}{2}h_{b_1} = \frac{1}{2} \cdot 1 + \frac{1}{2}h_{b_1} & (a_2 : H \rightarrow A, T \rightarrow b_1) \\
 h_{a_1} &= \frac{1}{2}h_{a_2} + \frac{1}{2}h_{b_1} & (a_1 : H \rightarrow a_2, T \rightarrow b_1) \\
 h_{a_0} &= \frac{1}{2}h_{a_1} + \frac{1}{2}h_{b_1} & (a_0 : H \rightarrow a_1, T \rightarrow b_1)
 \end{aligned}$$

2.3.4 Step 4: solve

From $h_{b_1} = \frac{1}{2}h_{b_2} + \frac{1}{2}h_{b_1}$, subtract $\frac{1}{2}h_{b_1}$: $\frac{1}{2}h_{b_1} = \frac{1}{2}h_{b_2}$, so $h_{b_1} = h_{b_2}$. Substituting into $h_{b_2} = \frac{1}{2}h_{b_1}$ gives $h_{b_1} = \frac{1}{2}h_{b_1}$, hence $h_{b_1} = 0$ and $h_{b_2} = 0$. Going up:

$$h_{a_2} = \frac{1}{2} + \frac{1}{2} \cdot 0 = \frac{1}{2}, \quad h_{a_1} = \frac{1}{2} \cdot \frac{1}{2} + 0 = \frac{1}{4}, \quad h_{a_0} = \frac{1}{2} \cdot \frac{1}{4} + 0 = \frac{1}{8}.$$

The start is a_0 , so

$$\mathbb{P}(\text{HHH first}) = \frac{1}{8}, \quad \mathbb{P}(\text{THH first}) = \frac{7}{8}.$$

$h_{b_1} = h_{b_2} = 0$ fully explains the spoiler

That $h_{b_1} = h_{b_2} = 0$ is striking: **once the game reaches “a T has appeared” (state b_1), the probability that HHH wins is exactly 0.** Reason: A would now need three H' s in a row, but before completing them a T almost surely appears first, and that T sends the state right back to b_1 while handing THH a perfect opening. So A' s only path to victory is **HHH right at the start** — probability exactly $\frac{1}{8}$. The $\frac{1}{8}$ is not a coincidence; it is the mathematical image of “must do it in one shot, or never recover.”

Two iron rules to take from this chapter

- (1) **Win-rate $\neq$ expected wait.** $\mathbb{E}[T_{\text{HHH}}] = 14$ says A waits long on average; but in the duel it' s a $\frac{1}{8}$ rout. “How fast you wait” and “who wins the duel” are different things.
- (2) **The joint chain is always reliable.** Take the state as a progress pair (i, j) , write the homogeneous equations $h_s = \frac{1}{2}h_{s'} + \frac{1}{2}h_{s''}$, and solve the linear system — this works for any two patterns, unequal length included. Every “pretty formula” later is checked against it.

3 The Crash Log: A Dead End in Search of a “Closed Form”

The joint chain solves any duel, but it requires **re-deriving a linear system for every pair**. The mathematician' s instinct: is there a **closed form** — plug in any X, Y and read off the win-rate? That is exactly what Conway' s correlation-number formula does.

This chapter records our own attempt to derive that closed form from first principles, **with every wrong turn. Spoiler: we went down a dead end, spent seven or eight hours, and were refuted by one line of code.** But read each detour — they are the real meat. Each maps to a conceptual error that' s very easy (for humans and AIs alike) to make.

3.1 Detour 1: Is $P(\text{length} = n) = \frac{1}{2^n}$?

Attempt

Split by “when the game ends” : $\mathbb{P}(X \text{ first}) = \sum_n P(\text{length} = n) \cdot P(X \text{ wins} \mid \text{length} = n)$ with $P(\text{length} = n) = \frac{1}{2^n}$.

Crash: confusing “one path’ s probability” with “an event’ s probability”

$\frac{1}{2^n}$ is the probability of one specific length- n sequence. But “the game ends at step n ” is a **whole family** of sequences. Its probability is (number of such sequences) $\times \frac{1}{2^n}$, never $\frac{1}{2^n}$. This is the “**specific string vs. event**” trap from Part I in new clothes: pinning one specific string costs $\frac{1}{2^{\text{length}}}$; an event made of many strings must sum them.

Click: both sample-space models are valid — just choose clearly

Two legitimate models:

- **Infinite-stream model:** sample space $\{H, T\}^\infty$. “ X first wins at position n ” is an unconditional event E_n^X , with probability = $\frac{\text{\#valid finite prefixes}}{2^n}$ (the infinite tail cancels). **No $P(n)$ factor here.**
- **Length-conditioned model:** choose length first, then condition; now $P(n) = \frac{1}{2^n}$ stays.

Both compute the same joint probability $\mathbb{P}(\text{stop at } n \text{ and } X \text{ wins})$, as “joint” vs. “marginal $\times$ conditional” — equal in value. We pick the infinite-stream model: $\mathbb{P}(X \text{ first}) = \sum_{n \geq |X|} \mathbb{P}(E_n^X)$, $E_n^X = \{T_X = n < T_Y\}$. This step **is correct** and survives all the way into the final solution.

3.2 Detour 2: The survival term forgot the opponent Y

Attempt

Mimic Part I’ s single-pattern recursion: $\mathbb{P}(E_n^X) = (\text{tail factor}) \times (1 - \sum_{k < n} \mathbb{P}(E_k^X))$, treating $1 - \sum \mathbb{P}(E_k^X)$ as “ X hasn’ t appeared yet.”

Crash: single-pattern “survival” drops half the condition

$1 - \sum_{k < n} \mathbb{P}(E_k^X)$ is only “ X hasn’ t won.” But E_n^X requires that both X and Y haven’ t won before step n . **It forgot Y :** it never removed the paths where Y already won. The correct “survival” is **joint survival** (neither X nor Y has appeared), and joint survival **cannot** be factored into “ X absent” $\times$ “ Y absent” — they share one coin stream and are strongly coupled.

What coupling means

“No HHH in the first 10 steps” and “no THH in the first 10 steps” use the same coins: more T’ s make HHH harder (good) and THH easier (bad). The two events are **oppositely entangled** by one stream, hence never independent. This is why the state must bind both progresses — again pointing at the joint state (i, j) .

3.3 Detour 3: The (k, p) partition is not mutually exclusive (the “compatible” trap)

Attempt

Partition by (k, p) : k = how many of Y' 's leading characters matched, p = where that match sits. Then $\mathbb{P}(E_n^X) = \sum_k \sum_p \mathbb{P}(E_n^X, Y' \text{ s first } k \text{ at position } p)$.

Crash: these events are “compatible” ; summing double-counts

One sequence can satisfy several (k, p) at once. E.g. THT for $Y = \text{THH}$ has a “T” at position 1 ($k=1, p=1$), another at position 3 ($k=1, p=3$), and “TH” at positions 1–2 ($k=2, p=1$). **The same sequence is counted by several terms.** These (k, p) events are **compatible** (can co-occur), **not mutually exclusive**. Additivity $\mathbb{P}(\bigcup E_i) = \sum \mathbb{P}(E_i)$ holds only for mutually exclusive events, so $\sum_{k,p}$ **double-counts**.

Mutually exclusive vs. compatible, bluntly

Mutually exclusive: cannot co-occur (“die shows 1” and “die shows 2”); probabilities add directly. **Compatible:** can co-occur (“even” and “> 3,” since 4 and 6 satisfy both); adding double-counts the overlap. Our (k, p) are compatible — a sequence can have several scattered partial matches of Y , so they get re-counted.

Click: use “the current progress now,” which is automatically exclusive

The second index must be a quantity with a unique value per sequence. That is: Y' 's current progress at the moment we enter the final stretch. With $m = n - |X|$,

$$\Gamma_m = \max\{j : 0 \leq j \leq |Y| - 1, \omega_{m-j+1..m} = Y_{1..j}\}.$$

Γ_m is unique per sequence (a single longest-match length), so partitioning by $\Gamma_m = j$ is **mutually exclusive**: $\mathbb{P}(E_n^X) = \sum_{j=0}^{|Y|-1} \mathbb{P}(E_n^X, \Gamma_m = j)$. In one line: **“where Y' 's prefix appeared in history” (many positions, compatible) vs. “ Y' 's progress now” (unique, exclusive).** The former is wrong, the latter right. The essence of a Markov state is drop history, keep only the current progress.

3.4 Detour 4: A “pretty-looking” self-built recursion

With the exclusive Γ_m we built a self-recursion that felt one step away from Conway:

1. **Layer 1 (time):** $\mathbb{P}(X \text{ first}) = \sum_n \mathbb{P}(E_n^X)$, by X' 's first-win position; exclusive. ✓
2. **Layer 2 (progress):** $\mathbb{P}(E_n^X) = \sum_j \mathbb{P}(E_n^X, \Gamma_m = j)$, by Γ_m ; exclusive.
3. **Segmentation:** $m = n - |X|$; prefix segment $1..m$; final stretch $m+1..n$ **pinned** to X , contributing $\frac{1}{2^{|X|}}$.
4. **Role rotation** (the seed of the crash): treat “prefix ending in Y' 's first j chars $Y_{1..j}$ ” as an isomorphic subproblem — the residual prefix $Y_{1..j}$ **promotes to the new**

pursued target, the original X **demotes to opponent**, giving the self-recursion

$$V(X, Y) = \frac{1}{2^{|X|}} \sum_{j=0}^{|Y|-1} \chi_j(X, Y) V(Y_{1..j}, X), \quad \chi_j(X, Y) = \mathbb{P}[X_{|X|-j+1..|X|} = Y_{1..j}].$$

Why it felt right — that “sense of unity” was real

Following this recursion we ourselves bumped into: a recursive invariant; X, Y swapping roles like turns in a game; the χ_j overlaps looking exactly like the embryo of a correlation number $L(X, Y)$. We strongly sensed a “conserved, gambling-like structure” behind it — the **smell of a martingale**. That experience of smelling unity from your own derivation is precious and real — **even though this particular recursion turns out wrong**. In math, “almost touching it” is itself nourishment.

3.5 Detour 5: Claude gives a false formula from memory, and calls it “obvious”

To connect the χ_j sum to “the classic Conway formula,” Claude (the AI writing this) gave, **from training memory**, a definition of a correlation number L and a four-term difference formula

$$\frac{\mathbb{P}(A \text{ first})}{\mathbb{P}(B \text{ first})} \stackrel{?}{=} \frac{L(B, B) - L(B, A)}{L(A, A) - L(A, B)},$$

asserting an identity $K(A, B) = L(B, B) - L(B, A)$ was “**obvious**.”

Crash: the definition and the “obvious” are both wrong

Claude’s L had the wrong weight direction, range, and numerator/denominator order; the “obvious” identity is a **flat-out false statement**. This was a **fuzzy memory from training data taken as fact** — the classic AI failure: treating “I think it’s like this” as “I proved this.”

GPT refutes it on the spot with a two-character counterexample

We handed the “identity to prove” to another AI (GPT). Instead of pushing forward it found a **counterexample**: take the shortest $A = \text{HH}$, $B = \text{HT}$:

$$K(\text{HH}, \text{HT}) = 3, \quad L(\text{HT}, \text{HT}) - L(\text{HT}, \text{HH}) = 2 - 0 = 2.$$

$3 \neq 2$. The “obvious” identity is refuted on the spot. On HHH/THH both sides happened to match, pure coincidence — exactly what gave Claude false confidence. **Memory-driven “assertion” vs. counterexample-driven “refutation”** : the latter wins outright.

A methodology note (first pass; reinforced later)

When anyone — an AI, you yourself — calls a formula “obvious” without proving it, the most effective response is not to believe but to find a **counterexample**, ideally the smallest, most special case (here, length 2). One counterexample topples an “obvious.” This is why we cultivate the habit of “small concrete examples + code,” not “it looks right.”

4 The Verification Crisis: A Code Referee Settles It

We already knew the four- L formula was refuted. But what about the recursion we ourselves derived, $V(X, Y) = \frac{1}{2^{|X|}} \sum_j \chi_j V(Y_{1..j}, X)$? It gave the correct $\frac{1}{7}$ odds on HHH/THH, so we assumed the recursion was fine and only the L -connection was misremembered. **That assumption was also wrong.**

4.1 Testing with the referee: an unequal-length example

Why we must use an unequal-length example

HHH and THH are both length 3 with very special overlap structure (one extreme, one trivial). **Many wrong formulas happen to work on such extreme cases.** To really test a formula, use structurally different examples — especially **unequal length**, since our derivation allowed unequal length from the start, the dimension most likely to expose a bug.

Take $A = \text{HH}$ (length 2), $B = \text{HTH}$ (length 3). First the referee (joint chain), then our recursion.

4.1.1 The referee gives the true value

The joint chain (solved exactly in code, cross-checked by Monte Carlo):

$$\mathbb{P}(\text{HH before HTH}) = \frac{2}{3}, \quad \text{odds} = 2 : 1.$$

4.1.2 What our recursion gives

Hand-evaluating the recursion:

$$V(\text{HH}, \text{HTH}) = \frac{1}{2^2} [\chi_0 V(\emptyset, \text{HH}) + \chi_1 V(\text{H}, \text{HH}) + \chi_2 V(\text{HT}, \text{HH})].$$

Here $\chi_0 = 1$; $\chi_1 = \mathbb{P}[\text{H} = \text{H}] = 1$; $\chi_2 = \mathbb{P}[\text{HH} = \text{HT}] = 0$. With $V(\emptyset, \text{HH}) = 1$ and $V(\text{H}, \text{HH}) = \frac{1}{2}[V(\emptyset, \text{H}) + V(\text{H}, \text{H})] = \frac{1}{2}[1 + \frac{1}{2}] = \frac{3}{4}$,

$$V(\text{HH}, \text{HTH}) = \frac{1}{4} [1 + \frac{3}{4}] = \frac{7}{16} = 0.4375.$$

Crash: recursion gives $\frac{7}{16}$, truth is $\frac{2}{3}$ — the recursion itself is wrong

$\frac{7}{16} = 0.4375 \neq 0.6667 = \frac{2}{3}$. Far off. So the problem is not “the last L -step” but the recursion itself. We thought passing HHH/THH ($\frac{1}{8}$) settled it — but that pair is too special and, once again, accidentally agreed, fooling us all night.

The one rock-solid fact — the touchstone hereafter

Three **fully independent** methods (joint-chain linear solve, Monte Carlo, and the matrix closed form below) **agree**:

$$\mathbb{P}(\text{HH before HTH}) = \frac{2}{3}.$$

Any formula or recursion claiming to solve this must reproduce $\frac{2}{3}$. Anything correct on HHH/THH but unable to reproduce $\frac{2}{3}$ on HH vs HTH is judged wrong.

5 The Correct Solution: State, Matrix Closed Form, and the Martingale

GPT reviewed our whole derivation, pinpointed **two substantive errors**, and gave a corrected version. **The corrected matrix closed form is code-verified correct** for equal and unequal length $(\frac{1}{8}, \frac{7}{8}, \frac{2}{3}, \frac{1}{3}, \dots)$.

5.1 Error 1: $\Gamma_m = j$ is not a sufficient state

The trouble is at the “boundary crossing”

Recall the segmentation: prefix $1..m$, final stretch $m+1..n$ pinned to X . We thought tracking “ Y ’s progress j on entering the stretch” ($\Gamma_m = j$) sufficed. But the prefix’ s tail **glued to** X ’ s leading chars may prematurely form X or Y before reaching position n ! To judge “does X form early,” knowing Y ’ s progress j isn’ t enough — you also need **how much the tail matches X** (progress i). The sufficient state is the **configuration** (i, j) .

Root cause: $\Gamma_m = j$ is “one-sided progress,” dropping the X dimension

$\Gamma_m = j$ records only Y ’ s progress. But the final stretch is X ’ s characters, which at the gluing point can both extend Y and complete X early. To handle “boundary premature termination” the state must record both X progress i and Y progress j . In one line: **the sufficient state is the configuration (i, j) , not the one-sided $\Gamma_m = j$** — which is precisely the joint chain’ s state.

5.2 Error 2: role rotation mistakes “progress” for a “new pattern”

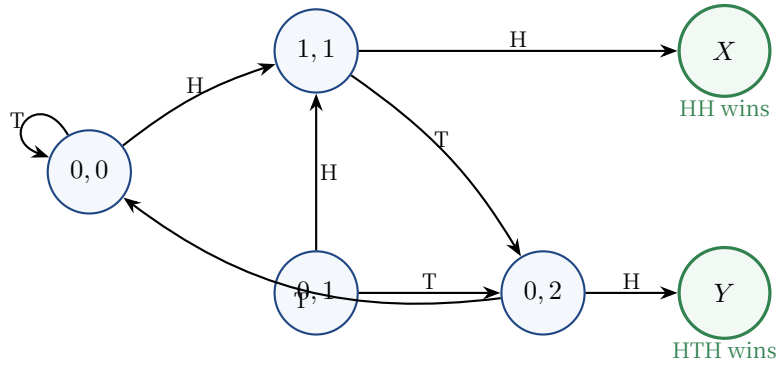
Root cause: $Y_{1..j}$ is not a new string, it is Y ’ s progress state

Our recursion wrote “ Y at progress j ” as the subproblem $V(Y_{1..j}, X)$, **treating the residual prefix $Y_{1..j}$ as a brand-new short pattern from scratch**. But “the complete short string $Y_{1..j}$ chased from zero” and “ Y already at j , continuing toward $|Y|$ ” are **not the same** — the latter carries the memory “ j matched,” which dictates where it falls back and whether it continues. We **used two complete strings as recursion parameters, losing the Markov progress memory**. That is why the recursion must collapse — each “rotation” silently forgets part of the state.

Click: the correct state was (i, j) all along — we looped back to the start

The correct recursion state must be the **progress configuration (i, j)** — exactly the joint Markov chain of Section 2. Our overnight self-built “two-complete-string recursion” looped all the way back: **the joint chain was right because its state keeps both progresses; our recursion lost the memory, hence wrong**. The error is **not** in the “two-layer partition” idea (that’ s right and valuable, and the correct solution keeps it) — it’ s in **secretly degenerating the state to “complete strings**.” A subtle, memorable slip: same word “recursion,” but state (i, j) is right, “complete string” is wrong.

5.3 A picture: the joint chain for HH vs HTH



(From $(0,0)$, states $(1,0)$ and $(1,2)$ are unreachable and omitted.) Arrows are labelled by the character read; reaching X means HH completed first, Y means HTH first.

Reading $\frac{2}{3}$ off the picture

At $(1,1)$, reading H wins for X directly. And $(0,0)$ reading H goes to $(1,1)$. So “two H’s up front” essentially locks in HH. Meanwhile HTH must first reach $(0,2)$ (i.e. produce HT) then an H — a longer path. Intuitively HH is favored; exactly, it’s $\frac{2}{3}$.

5.4 The matrix closed form: one formula for equal and unequal length

Let $\mathcal{S}$ be the set of non-absorbing configurations.

5.4.1 The safe-transition matrix M

What M counts

M records “from a configuration, reading one coin, where you can go without anyone winning.” Since each configuration has one H and one T edge, the number of safe ways from s to t is 0, 1, or 2.

$$M_{s,t} = \#\{a \in \{H, T\} : s \xrightarrow{a} t \text{ with no one winning}\} \in \{0, 1, 2\}.$$

The true transition is $M/2$. For the touchstone HH vs HTH, ordering states $(0,0), (0,1), (0,2), (1,0), (1,1), (1,2)$,

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

5.4.2 Counting prefixes: $N_m = e_0 M^m$

Why powers of M count “safe prefixes”

M counts “one safe step.” So an entry of M^m counts the number of paths that walk m steps safely from start to a given configuration. With each path’s probability 2^{-m} , this gives the probability that a length- m safe prefix lands in each con-

figuration.

With e_0 the unit row vector of the start $(0,0)$: $N_m = e_0 M^m$.

5.4.3 The final-sprint filter R_X : replacing the wrong χ_j

Why χ_j fails and a “dynamic filter” is needed

Our χ_j was a **static overlap indicator**. What we actually need: from a configuration s , **force the whole X onto it** — does it form X or Y midway? Only “safe midway, X absorbs at the last char” is legal. This requires **checking the whole X position by position**, not a static overlap.

Define $R_X(s) \in \{0,1\}$:

$R_X(s) = 1 \iff$ forcing $x_1 \cdots x_{|X|}$ from s : first $|X|-1$ steps no one wins, last char absorbs X .

For the touchstone, $R_X = (1, 1, 0, 0, 0, 0)^\top$.

5.4.4 Putting it together: the closed form

Summing “safe-prefix count $\times$ tail factor $\times$ sprint filter” over all m :

$$\mathbb{P}(X \text{ first}) = \sum_{m \geq 0} 2^{-(m+|X|)} e_0 M^m R_X = 2^{-|X|} e_0 \left(\sum_m (M/2)^m \right) R_X.$$

Since the game eventually absorbs, the geometric series converges to $(I - \frac{M}{2})^{-1}$:

$$\mathbb{P}(X \text{ first}) = 2^{-|X|} e_0 \left(I - \frac{M}{2} \right)^{-1} R_X$$

and symmetrically $\mathbb{P}(Y \text{ first}) = 2^{-|Y|} e_0 (I - \frac{M}{2})^{-1} R_Y$.

Code verification: this closed form is correct

For $X = \text{HH}$, $Y = \text{HTH}$: $(I - \frac{M}{2})^{-1}$'s first row is $(\frac{8}{3}, 0, \frac{2}{3}, 0, \frac{4}{3}, 0)$, dotted with $R_X = (1, 1, 0, 0, 0, 0)^\top$ gives $\frac{8}{3}$, times $2^{-2} = \frac{1}{4}$ gives $\frac{2}{3}$. ✓ **Verified by code over many pairs, equal and unequal length.** This is the true closed form, and it shows our χ_j error: it should have been the dynamic R_X , not the static χ_j .

5.5 Where exactly the martingale is

This is where GPT went deeper than we had. We had circled around “should one chase a martingale,” “did Conway just dress up a martingale,” without pinning it down. Here it is.

5.5.1 Three easily-confused objects

One line to separate them

R_X is a 0/1 “can you legally sprint” test; $W(s)$ is the probability of “a fresh complete X in the future”; $F(s)$ is the true probability “ X wins first from here.” Three distinct layers; conflate them and all is lost.

- $R_X(s)$: 0/1 filter, “is a full- X sprint from s legal.” Not a probability.
- $W(s) = \sum_m 2^{-(m+|X|)} e_s M^m R_X$: the probability of a fresh complete X as the closing block. **It misses “short completions”**: if s already has part of X (e.g. $X = \text{HH}$, s holds one H , next H wins immediately), that winning path is real but is not “a fresh complete X .” So $F(s) = B(s) + W(s)$ with $B(s)$ the short-completion terms.
- $F(s) = \mathbb{P}_s(T_X < T_Y)$: the **true absorption probability** that X wins first from s . This is the value function — the h_s of Section 2.

5.5.2 F satisfies Bellman; $F(S_{t \wedge \tau})$ is a strict martingale

For non-absorbing s , branching on the next coin (direct X -win contributes 1, Y -win 0, non-absorbing t contributes $F(t)$):

$$F(s) = b_X(s) + \frac{1}{2} \sum_t M_{s,t} F(t), \quad b_X(s) = \frac{1}{2} \# \{a : s \xrightarrow{a} X \text{ absorbs}\},$$

with $F(X \text{ win}) = 1$, $F(Y \text{ win}) = 0$.

This is Part I’ s first-step analysis, upgraded to two patterns

“Current win-rate = average of win-rates after each next coin.” Same as Part I’ s **first-step analysis**, i.e. the **Bellman expectation equation** (policy evaluation in dynamic programming). Note **no** max — coins are random, there’ s no decision, so it’ s pure expectation, not optimization.

Let S_t be the step- t configuration and $\tau = T_X \wedge T_Y$. Bellman says

$$\mathbb{E}[F(S_{(t+1) \wedge \tau}) \mid S_{t \wedge \tau} = s] = F(s) \Rightarrow \boxed{F(S_{t \wedge \tau}) \text{ is a strict martingale.}}$$

A martingale, in gambling terms

A **martingale** is the “current worth” of a fair game: wherever you are, the expected next worth equals the current worth (no edge). Here $F(s)$ = “probability X ultimately wins from the current configuration,” and it has exactly this property: one more coin, and on average the win-rate is unchanged (though it jumps up or down on any single path). **It is the conditional mean that’ s fixed, not each path.**

5.5.3 Odds is the ratio of two martingales’ initial values

With $F_X(s) = \mathbb{P}_s(T_X < T_Y)$, $F_Y(s) = \mathbb{P}_s(T_Y < T_X)$, both martingales along $S_{t \wedge \tau}$, ending in the indicators “ X wins” / “ Y wins”:

$$\frac{\mathbb{P}(X \text{ first})}{\mathbb{P}(Y \text{ first})} = \frac{F_X(s_0)}{F_Y(s_0)}.$$

Key clarification: ratio of initial values, not of processes

$\frac{F_X(S_t)}{F_Y(S_t)}$ is generally not a martingale. The correct statement: odds is the ratio of the two martingales' values **at the initial configuration** s_0 . Our earlier “two bettors vs. one new bettor per step” confusion stemmed from blurring “martingale process” and “martingale initial value.”

Revisiting all our martingale confusion

“Should rational gambling chase a martingale,” “did Conway dress up a martingale” — now fully clear. **The martingale is not** the “bankroll $W(s)$ ” or the “filter R_X ” we kept staring at, **but the absorption-probability value function** $F(s)$ (the h_s of the joint chain). It solves the Bellman equation, and $F(S_{t \wedge \tau})$ is a strict martingale. The equal-length Conway formula is exactly **this martingale structure collapsing onto overlap numbers** — the martingale is not a veneer; it's the machine that compresses the whole configuration DP into an overlap difference. We never “got the unity” before because we **stared at the wrong object** (W/R_X instead of F).

6 Three Cases and That Pretty Conway Formula

The matrix closed form holds always, but by length relation there are three tiers, and the equal-length tier collapses to something especially pretty.

6.1 Case 1: identical patterns

If $X = Y$ then $T_X = T_Y$, “ X strictly before Y ” is impossible, $\mathbb{P}(T_X < T_Y) = 0$. So Penney's game defaults to $X \neq Y$.

6.2 Case 2: equal length, different — the pretty Conway formula

Why equal length “collapses” to a simple formula

When $|X| = |Y| = n$, the nuisance of “boundary premature termination” doesn't break symmetry, and the matrix inverse $(I - \frac{M}{2})^{-1}$ simplifies to pure “overlap number” arithmetic — yielding Conway's formula in terms of self- and cross-overlaps alone.

Define the **Conway correlation number** for equal-length U, V :

$$C(U, V) = \sum_{k=1}^n \mathbb{1}[U' \text{ s last } k = V' \text{ s first } k] \cdot 2^{k-1}.$$

$C(U, U)$ is **self-overlap**, $C(U, V)$ **cross-overlap**. Then

$$\frac{\mathbb{P}(X \text{ first})}{\mathbb{P}(Y \text{ first})} = \frac{C(Y, Y) - C(Y, X)}{C(X, X) - C(X, Y)}$$

What numerator and denominator measure

$C(X, X) - C(X, Y)$ = “how like itself X is” minus “how like the opponent X is.” More self-overlap (X snags on its own tail, harder to complete) hurts it; more overlap with the opponent (X ’s tail feeds the opponent) also hurts it. **This “self-similarity minus cross-similarity” difference exactly captures a pattern’s strength** — the algebraic incarnation of “self-overlap is the engine.”

Verification ($A = \text{HHH}$, $B = \text{THH}$, weights 2^{k-1}):

$$C(A, A) = 7, \quad C(B, B) = 4, \quad C(A, B) = 0, \quad C(B, A) = 3,$$

$$\frac{\mathbb{P}(A \text{ first})}{\mathbb{P}(B \text{ first})} = \frac{C(B, B) - C(B, A)}{C(A, A) - C(A, B)} = \frac{4 - 3}{7 - 0} = \frac{1}{7} \Rightarrow \frac{1}{8} : \frac{7}{8}. \quad \checkmark$$

6.3 Case 3: unequal length — do not force the Conway difference

The root cause of our whole overnight crash

When $|X| \neq |Y|$, **you cannot use the equal-length Conway difference**. Extreme example: $X = \text{HH}$, $Y = \text{HHH}$. Whenever HHH appears, HH has already appeared, so Y can never strictly precede X ($\mathbb{P}(Y \text{ first}) = 0$). The general solution is the **matrix closed form** $2^{-|X|} e_0 (I - \frac{M}{2})^{-1} R_X$, which collapses to Conway only when equal. Our self-built derivation allowed unequal length from the start (correct, more general), but the “classic Conway formula” Claude pulled from memory is equal-length only — **measuring an unequal cloth with an equal ruler**: the root of the seven-or-eight-hour crash.

A human vigilance that saved the day

It was Bo’s repeated insistence that “**we are deriving the unequal case**” that finally punctured the illusion — it forced out the distinction “equal-length Conway doesn’t apply to unequal.” Amid sources all confidently outputting (AI memory included), insisting on one unglamorous structural fact (lengths may differ) is often the very needle that pops the balloon.

6.4 Aside: the discarded K vs. the true Conway C

- Our $K(X, Y) = \sum_{k=0}^{|Y|-1} c_k 2^k$ (with the $k=0$ empty-overlap term): definable as an overlap score, but $\frac{K(A, B)}{K(B, A)}$ is **not** the correct odds, not even for equal length (HH vs HT is a counterexample: K gives 1 : 1). **Discard**.
- The true Conway C (weight 2^{k-1} , k from 1, difference form): correct for equal length, code-verified.
- Unequal length: no simple binary overlap ratio; use the matrix closed form.

7 Back to Question 2: No Strongest Pattern (Non-transitivity)

Computing length-3 patterns pairwise with any correct tool reveals a startling cycle:

$$\text{HHH} \prec \text{THH} \prec \text{TTH} \prec \text{HTT} \prec \text{HHH},$$

each “ $\prec$ ” meaning “the latter beats the former by $\geq 2 : 1$.” **It closes into a loop — like rock-paper-scissors, no pattern is strongest.**

So “pick the shortest expected wait” is not a winning strategy

This is a **minimax / non-transitive** structure: whatever you pick first, the opponent, seeing your choice, picks one tailored to beat it (intuitively, prepend “almost your opening” to cut you off). Going first is a disadvantage; there is no invincible pattern. The engine is again the **asymmetry of self-overlap** — the $\mathbb{E}[T_{HHH}]=14$ vs $\mathbb{E}[T_{HTH}]=10$ gap from Part I grows here into a “nobody is strongest” loop. One thread (self-overlap) runs through both parts.

Why this is so counterintuitive and so important

Our brains default to “strength lies on a line” (transitivity): if A beats B and B beats C, then A beats C. Penney’s game is a clean **counterexample**: the beats-relation can cycle. Such non-transitivity is everywhere — voting paradoxes, ecological rock-paper-scissors, certain dice games. Penney demonstrates it with the simplest possible coin.

8 Meta-Lesson: Approaching Truth When Every Source Errs

This whole crash is a complete case study in **multiple AIs poking holes in each other plus a code referee**. For anyone who will do research or use AI tools, this methodology is **worth more than Conway’s formula itself**.

8.1 The full “crash-and-rescue” chain

1. **Claude gives a wrong L from memory** $\rightarrow$ Bo, trusting it, spends half a day proving an identity that cannot hold.
2. **GPT refutes the false identity** with the two-character counterexample HH vs HT.
3. **Our K -ratio “verified $\frac{1}{7}$ on HHH/THH”** gave false confidence.
4. **The code joint chain (referee), on the unequal HH vs HTH**, settles it: K wrong, the recursion itself wrong, truth $\frac{2}{3}$.
5. **GPT’s review** pinpoints the two substantive errors (insufficient state, role rotation) and gives the corrected matrix closed form and martingale.
6. **Claude re-verifies GPT’s closed form by code** (not by trust, by running it), confirming $\frac{1}{8}, \frac{2}{3}$ before writing it here.

8.2 Four transferable disciplines

Carve these in

(1) No single source is trustworthy alone — AI, “classic formulas,” your own intuition included. AIs have memory bias and will treat “I think it’s like” as “I proved it.” Anything “obvious” or “the classic formula is just this” without on-the-spot derivation or a lookup: **doubt by default**.

(2) **Single-point verification gives false confidence.** One example passing $\neq$ formula correct. HHH/THH is too special and fooled two wrong formulas. **Use several structurally different examples, especially unequal length.**

(3) **Code (or independently computable ground truth) is the final referee.** Pretty self-built derivations, remembered classics, and “it feels unified” can all deceive; **things that produce numbers do not.** Build the habit of “derive, then write a tiny program to check.”

(4) **Cross-refutation beats solo deep-digging.** Let different sources (Claude / GPT / DeepSeek / code) find each other’s faults; truth surfaces in the collision. Here GPT refuted Claude, code refuted every formula, and the correct solution was forced out.

8.3 The mathematics that actually settled

Seven sentences to take away

1. **Win-rate $\neq$ expected wait.** Win-rate is an absorption probability (homogeneous equation, no +1); wait is an absorption time (inhomogeneous, +1). Decoupled.
2. **The state must be a progress configuration (i, j) , never degenerated to complete strings.** The hardest lesson here.
3. **“Current progress now” is exclusive (summable); “positions where it appeared in history” are compatible (double-count).** Γ_m right, (k, p) wrong.
4. **The martingale is the absorption-probability value function $F(s)$ ($= h_s$), not the bankroll W or the filter R_X .** $F(S_{t \wedge \tau})$ strict martingale; odds is the ratio of two martingales’ initial values (not of processes).
5. **Equal length: the pretty Conway difference; unequal: the general matrix closed form.** Don’t measure unequal cloth with an equal ruler.
6. **Ground truth (joint chain + code) is the final referee;** single-point checks, remembered “classics,” and pretty self-recursions can all deceive.
7. **Self-overlap is the engine of non-transitivity** — it lets the “no strongest pattern” loop exist, threading Parts I and II together.

8.4 Directions for a Part III

1. **Collapse the matrix closed form to equal-length Conway:** explicitly show that for $|X| = |Y| = n$, $e_0(I - \frac{M}{2})^{-1}R_X$ reduces to $C(Y, Y) - C(Y, X)$.
2. **Build Conway directly from the martingale:** from “ $F(S_t)$ is a martingale,” find the explicit harmonic function (constructed from overlap numbers when equal-length) — the standard “martingale proof of Penney.”
3. **The physical meaning of L as cross-correlation / convolution** (not developed here): $C(X, X)$ is autocorrelation (a pattern’s echo against itself), $C(X, Y)$ cross-correlation; formally the correlation polynomial (Guibas–Odlyzko), the same object as the correlation function in signal processing and the similarity score in retrieval. Worth its own chapter.

Acknowledgments and attribution. The main derivation, all the wrong turns, and the decisive vigilance

that rescued it (insisting on unequal length, distrusting single-point checks) were driven by Bo; Claude judged steps adversarially, produced this write-up, and contributed one textbook-grade cautionary tale (a wrong L from memory, declared “obvious”); GPT contributed the counterexample refutation and the corrected review (matrix closed form, exact location of the martingale); the Python joint chain served as ground-truth referee. This is one real run of the multi-AI cross-review methodology — and, beyond the mathematics, the thing this log most wants to convey.